

Learn mode

Claude does the work. You learn from it while it happens.

Arvind · ant-takehome.vercel.app · github.com/arvindm1991/ant-takehome

THE PROBLEM

The work still gets done. The skill doesn't get built.



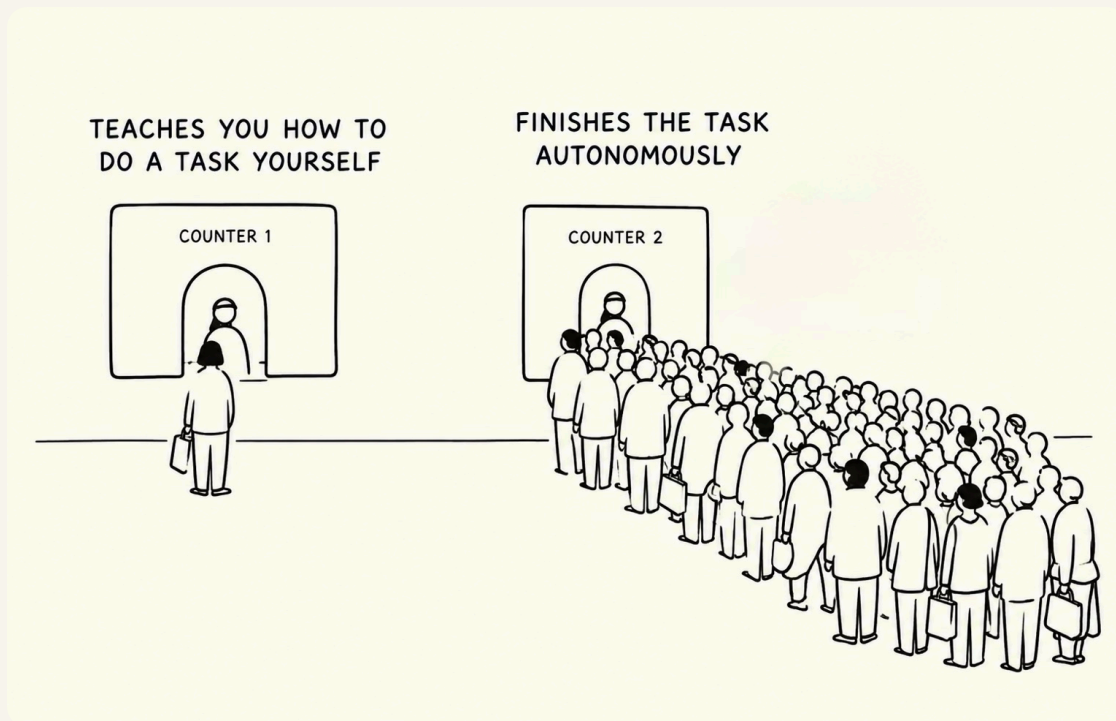
Skill comes from the friction of doing the work. Agents remove that friction by default: the weight gets lifted and the meal gets made, and the muscle, the knives and the pots go unused. Showing more steps doesn't fix it. Claude already shows its reasoning and code, and people skip to the answer.

The upside is that this isn't learning for its own sake. The task is real, the user just asked for it, and an expert is doing it right now. That is the best moment to learn, like training on a shop floor while a master does the job.

We need to put back just enough friction, while the work is being done, without slowing the work down.

WHY OPTION B, AND WHY INSIDE CLAUDE

People queue at the counter that finishes the task



When something needs doing, people go where it gets done quickly. That's why so many good learning products get little use; I've built some of them. A separate, high-friction learning app is counter 1: good for the few who choose it, empty for everyone else.

So I didn't build counter 1. **Learn mode puts the teaching at counter 2**, inside the Claude session where the queue already is, and it never makes the task wait. That is Option B, turning AI-assisted work into learning, placed where it can actually reach people.

THE SOLUTION

A learning agent that watches Claude work and teaches you from it

Claude (main agent)

Does the task at full quality. Never knows Learn mode exists; no learner data enters its context.

→
read-only

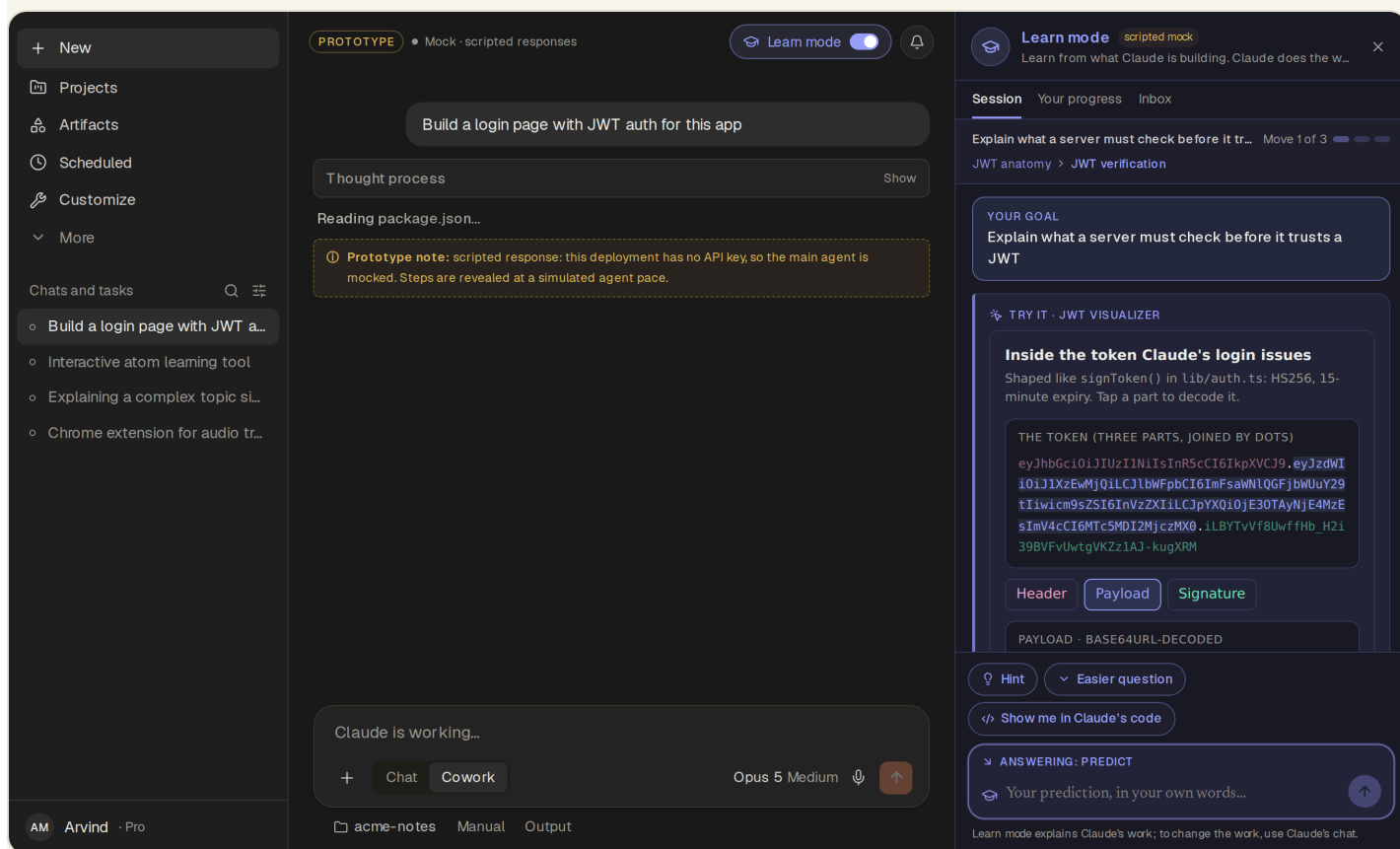
Learn mode (learning agent)

Reads Claude's plan, files and reasoning as they arrive. Teaches from them, grounded in Claude's actual code.

ONE JOURNEY: "BUILD A LOGIN PAGE WITH JWT AUTH FOR THIS APP"

- 1 **Turn on Learn mode** (or tap the chip under a task). Claude starts working; you are never in its way.
- 2 **Pick what to learn.** Goal cards come from the task itself: where login plugs into this repo, what a server must check before it trusts a JWT, why bcrypt and not SHA-256.

- 3 **See it first.** An interactive built from Claude's code, such as a visualizer that decodes the exact token Claude's login will issue.
- 4 **Predict before Claude writes it.** "What must `verifyToken()` check?" When the file lands, Learn mode checks your prediction against the real code and links to the line.
- 5 **Go where you want.** Dig deeper, zoom out or try it hands-on; after a miss, get a hint, an interactive, or the exact spot in Claude's code.
- 6 **It comes back.** A recap shows how your mastery moved. Days later, or when a related task comes up, a one-question refresher brings the topic back.



The prototype: Claude building auth on the left; Learn mode on the right, opening with a token visualizer before asking for a prediction.

DESIGN AND PROTOTYPING PROCESS

From the obvious idea to the one that ships

- **Rejected the default.** Every frontier model I sparred with (Claude included) first proposed pausing the agent at decision points so the user steers. That caps quality at the learner's skill and fills the agent's context with guesses. The user rides in the front passenger seat, not the driver's.
- **Spec first.** A written spec with a numbered decision log, then milestones: shell and main agent, learning loop, memory, supporting moves, refreshers, ship.
- **Real Claude API throughout.** Main agent with streamed reasoning; a separate learning agent using tools; a small model deciding whether a task is worth learning from; interactives generated on demand.

- **Iterated on real use.** A blank chat box read as “a second Claude”, so it became goal cards and next moves. That felt like homework, so it became show-don't-tell with one text box of its own, and on means on. Then it was made to work on a phone.

HUMAN AGENCY

It enhances judgment instead of replacing work

- **Claude still does the task, fully.** Learning never blocks, slows or steers it.
- **You choose** whether to learn, what, and how deep. Everything can be dismissed, and Learn mode is one switch.
- **Understand before you review.** Knowing why Claude made each choice is what lets you oversee it and push back.
- **It never does the task for you.** It asks, shows and checks. It asks about Claude's choices rather than judging them, since it doesn't have Claude's full context.

LEARNING PRINCIPLES

What the design borrows from how people learn

PRINCIPLE	WHERE IT SHOWS UP
Apprenticeship	An expert does the real task while a coach explains, asks and checks.
Just-in-time relevance	Topics come from the task you just asked for, so there's no need to persuade you they matter.
Prediction and retrieval	You predict before the code exists, then compare against what Claude wrote.
Active over passive	Interactives you manipulate rather than read. A “watch the agent” mind map was dropped: it feels like teaching without evidence of learning.
Scaffolding	Hints, never answers; an easier question or an interactive after a miss.
Spacing and interleaving	Refreshers at growing intervals, and when a related task comes up.
A model of the learner	Mastery, evidence and misconceptions decide the next move and skip what you already know.

MEASURING SUCCESS

Measure learning, not engagement

- **North star:** first-try accuracy on refreshers taken days later, in a different task. That measures retention and transfer, not time spent.

- **Learning curve:** mastery over time per topic; misconceptions resolved.
- **Adoption:** offered → opened → first answer → session completed, compared across entry points (switch vs chip).
- **Guardrails:** no change to task completion time or satisfaction; switch-off rate; how often people type task requests into Learn mode's box.
- **Quality evals** for the learning agent: grounded in Claude's code, no spoilers, grader agrees with humans, shows before it tells.

SCALE

Built to reach millions without a new app

- **Distribution is built in:** it's a switch in Claude and Claude Code, not a destination.
- **Works in any domain:** topics come from the work itself, not a hand-built curriculum. The demo is code; the same loop fits writing and analysis.
- **Segments:** on by default for students (set by schools); off for professionals, with a light nudge budget; deeper stretch goals and review for power users.
- **Cost scales with engagement:** it runs only when on; cheaper models (Sonnet, Haiku) watch and decide; Opus builds interactives only when someone asks to see or try something; prompts put stable context first so it can be cached.
- **Rollout:** feature flag, then A/B on entry points, graduating on the north star with guardrails intact.

Prototype notes: the repository and agent pacing are simulated and labelled in the app; learner memory lives in the browser. Model calls are real when an API key is set.